Agent Self-Evolution

Phase 1 Validation Report

Date: June 25, 2026

Repository: github.com/jramos/agent-self-evolution

Executive Summary

Agent Self-Evolution is a standalone optimization pipeline that uses DSPy and GEPA (Genetic-Pareto Prompt Evolution) to automatically improve an agent's skills, tool descriptions, system prompts, and code through evolutionary search — all via API calls with no GPU training required. Originally built for Hermes Agent; now works for any agent framework that emits SKILL.md files.

This report documents the Phase 1 validation of the skill evolution pipeline. Using DSPy GEPA with OpenAI's gpt-4.1 / gpt-5-mini / gpt-4.1-mini stack, we evolved the **arxiv** skill end-to-end. The evolution achieved a **40.0% size reduction** AND a **+5.4 point holdout improvement** (0.908 → 0.962) — the paired-bootstrap 90% CI on the per-example diff is [+0.017, +0.095], excluding zero. The deploy gate accepted the candidate under the *non-inferiority* rule; the knee-point selector picked the highest-val candidate within the ε-band.

KEY RESULT — arxiv (Hermes Agent), GEPA light budget
Size: 10,036 → 6,017 chars (-40.0%)
Holdout score: 0.908 → 0.962 (Δ +0.054, 90% CI [+0.017, +0.095], n=65)
Decision: DEPLOYED (non-inferiority gate; bootstrap CI excludes zero)

Background

Hermes Agent is a general-purpose AI agent built by Nous Research that uses tool-calling LLMs to complete tasks via terminal commands, file operations, web search, code execution, and more. The Self-Evolution framework was originally built for it but has since been generalized: a pluggable `SkillSource` protocol now discovers skills in the Hermes Agent layout, the Claude Code plugin cache, or any flat local directory — the same optimizer can target any of them. Hermes Agent's behavior is governed by three layers:

Layer	What It Is	How It's Currently Improved
Model Weights	The underlying LLM (Claude, GPT, etc.)	RL training (Tinker-Atropos)
Instructions	Skills, system prompts, tool descriptions	Manual authoring (static)
Tool Code	Python implementations of each tool	Manual development

The **instructions layer** (highlighted) is the sweet spot for automated optimization: it's pure text that LLMs can meaningfully mutate, changes are immediately deployable, and results are directly measurable. Agent Self-Evolution targets this layer.

Approach: Evolutionary Optimization

Three Optimization Engines

Engine	What It Optimizes	License	Role
DSPy + GEPA	Skills, prompts, tool descriptions	MIT	Primary (validated)
DSPy MIPROv2	Few-shot examples, instruction text	MIT	Fallback optimizer
Iterative test-feedback repair	Tool implementation code	MIT	Code repair (Phase 4)

GEPA (Genetic-Pareto Prompt Evolution) is the star engine — an ICLR 2026 Oral paper from Stanford/UC Berkeley. Unlike traditional evolutionary search that only sees pass/fail scores, GEPA reads full execution traces to understand *why* things failed, then proposes targeted mutations. It outperforms reinforcement learning (GRPO) by +6% with 35x fewer rollouts, and outperforms DSPy's previous best optimizer (MIPROv2) by +10%. It works with as few as 3 training examples.

The Optimization Pipeline

1. **Discover and load skill** — Resolve the skill via the SkillSource protocol (Hermes / Claude Code / local-dir), parse YAML frontmatter and body
2. **Generate eval dataset** — An LLM (gpt-4.1-mini) reads the skill and synthesizes 60+ (task, expected_behavior) pairs, then splits into ~36% train / ~29% val / ~36% holdout
3. **Wrap as DSPy module** — The skill text becomes a parameterized DSPy module where the instructions are the optimizable parameter
4. **Run optimizer** — DSPy GEPA evolves the skill instructions, scored by an LLM-as-judge with a structured rubric (correctness, procedure-following, conciseness)
5. **Knee-point Pareto selection** — Among candidates within ε of the val-best, pick the smallest one (parsimony principle for deployable, cache-friendly skills)
6. **Validate constraints** — Static size limits, structural integrity, growth-quality gate backed by a paired-bootstrap CI on the held-out set
7. **Report** — Structured gate decision JSON, before/after artifacts, full LM trace log

Critically, **no GPU training is involved**. The entire pipeline operates via LLM API calls. The arxiv run reported here issued ~1,568 LM calls (1,526 to gpt-4.1-mini for synthesis / evaluation / judging, 42 to gpt-5-mini for GEPA reflection); typical end-to-end cost is $5–8 in OpenAI API credits per skill on the light GEPA budget at the current `eval_dataset_size=150` default.

Phase 1 Experiment

Configuration

Parameter	Value
Target Skill	arxiv (Hermes Agent — arXiv paper search and retrieval)
Baseline Size	10,036 characters
Optimizer LM	openai/gpt-4.1
Reflection LM (GEPA)	openai/gpt-5-mini
Eval / Judge LM	openai/gpt-4.1-mini
Optimizer	DSPy GEPA (light budget — ~440 metric calls)
Synthetic Eval Set	178 examples (63 train / 50 val / 65 holdout)
Total Optimization Time	3,353 seconds (~55 minutes)
Total LM Calls	~1,568 (1,526 gpt-4.1-mini + 42 gpt-5-mini)
Total Cost (USD)	$6.00
Quality Gate	non-inferiority (tolerance=0.02; regression-only branch)
Knee-point Strategy	val-best (default; smallest available via --knee-point-strategy)

Evaluation Dataset

The evaluation dataset was synthetically generated by openai/gpt-4.1-mini. Given the full arxiv SKILL.md text, the model produced 178 realistic test cases with rubric-based expected behaviors, then split them into train / val / holdout per the framework's configured ratios (the LM occasionally produces slightly more than requested). Examples drawn from the generated set:

Task Input	Expected Behavior (Rubric)
Fetch paper metadata with multiple categories and verify primary category identification.	Primary category is correctly identified and displayed separately.
Extract and parse XML output from arXiv API using provided python snippet for a keyword search.	Parsed output includes numbered list with paper IDs, titles, authors, published dates, categories, abstracts, and PDF links.
Search papers with comment containing 'accepted NeurIPS' to find accepted conference papers.	Returns papers with comments mentioning acceptance at NeurIPS, metadata included.

Fitness Function

Fitness is now measured by an LLM-as-judge (gpt-4.1-mini) that scores each candidate output along three independent rubric dimensions. The composite score is a weighted combination, with a length-penalty term that discourages runaway expansion:

```
composite = 0.5·correctness + 0.3·procedure_following + 0.2·conciseness –
                                length_penalty
```

The judge also returns a free-text feedback string that GEPA's reflection LM consumes to propose targeted instruction-text mutations on the next iteration — this trace-aware loop is the core of GEPA's sample efficiency. The keyword-overlap heuristic used in the original Phase 1 experiment was retired in favor of this rubric scorer.

Results

Metric	Baseline	Evolved (knee-point pick)	Δ
Body size (chars)	10,036	**6,017**	-40.0%
Avg holdout score (n=65)	0.908	0.962	+0.054
Bootstrap mean diff	—	+0.054	—
Bootstrap 90% CI lower	—	+0.017	—
Bootstrap 90% CI upper	—	+0.095	—
Decision	—	DEPLOYED	**CI excludes 0**

The evolved arxiv skill is **40.0% smaller** than baseline (6,017 vs 10,036 characters) AND scores **+0.054 higher** on the 65-example holdout (0.962 vs 0.908). The paired 90% bootstrap CI on the per-example difference is [+0.017, +0.095] — both ends positive, so we have ≥90% confidence the improvement is real, not sampling noise. The deploy gate passes under the new *non-inferiority* rule (introduced this cycle) and would have passed the older *no-regression-only* rule too because the bootstrap mean is positive. The candidate is written to `evolved_skill.md` alongside the verbatim baseline for diffing.

How the Result Was Produced

GEPA evolves skill instructions through a reflective loop:

1. Run candidate skill instruction text on training examples; the judge scores each output and emits free-text feedback
2. Reflection LM (gpt-5-mini) reads the execution traces + feedback and proposes a targeted mutation of the instruction text
3. Score the mutated candidate on the validation set (50 examples); track every candidate's per-example Pareto front
4. After GEPA's light-budget search (~440 metric calls), freeze the candidate population

5. Knee-point selection: among all candidates within ε = 1/n_val of the val-best, pick the highest-val candidate (smallest body as tiebreak). On this run candidate 7 won the band (val=0.980, rank 1 of 3, 5,754 body chars) and is also the candidate GEPA's own default selector would have chosen — the val-best and GEPA-default converged here.

Three configuration choices made this run deployable: (a) the `eval_dataset_size=150` default sized a 65-example holdout that's tight enough on the bootstrap CI to detect small effects; (b) the *non-inferiority* deploy gate ships compression-without-regression candidates explicitly — though this run cleared the stricter *no-regression-only* rule too; (c) knee-point selection picks the val-best candidate within the ε-band rather than the smallest body, so the pick is the candidate most likely to generalize.

Safety and Guardrails

Every evolved variant must pass all of the following constraints before deployment:

Constraint	Enforcement	Status
Self-evolution test suite	282 pytest tests pass on the optimizer itself	Implemented
Static size limits	Skills ≤15KB, tool descs ≤500 chars (configurable)	Implemented
Absolute char ceiling	Hard cap on evolved artifact size (default 5,000)	Implemented
Growth-quality gate	Required improvement scales linearly with growth %	Implemented
Paired-bootstrap CI	90% CI on per-example holdout diffs gates deploy	Implemented
Knee-point selection	Smallest candidate within ε of val-best	Implemented
Structural integrity	Valid YAML frontmatter required	Implemented
Deployment via PR	Human review required, never auto-merge	By design
Benchmark regression	TBLite / skill-specific harness must hold	Planned

Source skill repositories are never modified directly. All evolution output (evolved artifacts, gate decisions, run logs) is written under the framework's local `output/` directory, and improvements are proposed as pull requests against the source repo for human review.

Roadmap

Phase	Target	Engine	Timeline	Status

Phase 1	Skill files (SKILL.md)	DSPy + GEPA	3-4 weeks	Validated ✓
Phase 2	Tool descriptions	DSPy + GEPA	2-3 weeks	Validated ✓
Phase 3	System prompt sections	DSPy + GEPA	2-3 weeks	Validated ✓
Phase 4	Tool implementation code	Iterative test-feedback repair	3-4 weeks	Validated ✓
Phase 5	Continuous improvement	Propose-only triage sentinel	2 weeks	Shipped ✓

Each phase must demonstrate measurable improvement and pass benchmark regression gates before proceeding. If a phase does not produce meaningful gains, we reassess before continuing. The full plan is documented in PLAN.md within the repository.

Immediate Next Steps

1. **Evolve more skills** — Run the same pipeline against additional Hermes and Claude Code skills to measure how often the regression-free deployment criterion is actually met and where the framework's defaults need calibration.
2. **Calibrate the deploy gate** — Use the bootstrap CIs from a portfolio of runs to set evidence-based defaults for `growth_free_threshold` and `growth_quality_slope`, and revisit the knee-point ε choice (this run picked a candidate that was only 5% smaller than the val-best at meaningful val cost).
3. **Larger holdout sets** — n=23 left a wide bootstrap CI; raising `eval_dataset_size` or `holdout_ratio` would tighten the deploy / reject signal.
4. **Benchmark gating** — Add TBLite or skill-specific regression harnesses for skills where the bootstrap CI alone is too noisy.
5. **PR automation** — Auto-generate pull requests against the source skill repository with the evolved artifact, gate decision JSON, and full metrics.
6. **Tier 2: tool descriptions** — Extend the optimizer beyond skill files to tool description text (currently a stub).